SYSTEM ARCHITECT AUTOMOTIVE

GIÁO TRÌNH CHUYÊN SÂU AUTOSAR

CAN RX → COM

Luồng nhận dữ liệu từ CAN Bus đến Signal tầng ứng dụng

Biên soạn: **Phạm Văn Vũ**

HALA Academy

GIÁO TRÌNH CHUYÊN SÂU AUTOSAR CAN RX STACK

Từ CAN Bus đến `Com_ReceiveSignal()`, qua 4 tầng BSW

> Tài liệu này bám sát đúng code demo trong repository hiện tại. Mục tiêu không chỉ là mô tả "có các lớp AUTOSAR", mà là chỉ rõ **dữ liệu đang nằm ở đâu, ai gọi ai, và cuối cùng Application lấy được giá trị Signal bằng cách nào**.

Một câu chốt bản chất của luồng RX: Ứng dụng không trực tiếp đọc CAN frame. Khi CAN frame đến phần cứng, chuỗi callback tự động chạy từ `Can Driver → CanIf → PduR → COM` để copy dữ liệu vào *shadow buffer*. Sau đó, ứng dụng chỉ việc gọi `Com_ReceiveSignal()` để đọc Signal từ buffer đó.

Mục lục

1. Mục tiêu học tập và bức tranh toàn cục
2. CAN là gì ở góc nhìn kỹ thuật hệ thống
3. Kiến trúc AUTOSAR COM Stack và vị trí luồng RX
4. Mapping trực tiếp vào code và cấu hình của repo
5. Luồng thực thi đầy đủ từ CAN Bus đến Application
6. Phân tích ở mức byte/bit: Pack và Unpack Signal
7. Timeline thực tế — Theo dõi 1 CAN frame xuyên suốt
8. Sơ đồ UML chi tiết
9. So sánh luồng TX và RX
10. Kiểm chứng bằng log và lab mô phỏng
11. Các điểm dễ nhầm và kết luận kỹ thuật

---

1. Mục tiêu học tập và bức tranh toàn cục

Sau khi học xong tài liệu này, người học cần trả lời được các câu hỏi sau mà không còn mơ hồ:

- Tại sao Application không gọi `CAN_Receive()` trực tiếp mà phải qua 4 tầng BSW?
- Dữ liệu tồn tại lần lượt ở những bộ đệm nào từ khi nhận CAN frame đến khi Application đọc Signal?
- Sự khác nhau giữa **CAN ID** (`0x181`), **PDU ID** (`5`) và **Signal ID** (`17`) là gì?
- Vai trò của `Can` , `CanIf` , `PduR` và `Com` khác nhau ở chỗ nào?
- Vì sao luồng RX chia làm 2 giai đoạn riêng biệt (Nhận & Lưu vs Đọc Signal)?
- Signal 16-bit (RPM) được unpack từ buffer như thế nào?

> **Chuỗi đi đúng của dữ liệu trong luồng RX:**
> `CAN Bus → bxCAN FIFO → Can_MainFunction_Read() / ISR → CanIf_RxIndication() → PduR_CanIfRxIndication() → Com_RxIndication() → Shadow Buffer → Com_ReceiveSignal() → Application`

---

2. CAN là gì ở góc nhìn kỹ thuật hệ thống

2.1 Bản chất của CAN

CAN (Controller Area Network) là mạng truyền thông chính trong ô tô, dùng cho các hệ thống yêu cầu độ tin cậy cao và tốc độ trung bình như động cơ, phanh, hộp số.

Đặc tính	CAN	Ý nghĩa trong demo
Cấu trúc bus	Multi-master, event-driven	Bất kỳ Node nào cũng có thể gửi frame bất cứ lúc nào.
Cơ chế truy cập bus	CSMA/CA – Arbitration theo ID	ID càng nhỏ, ưu tiên càng cao.
Tầng vật lý	Differential 2-wire (CAN_H, CAN_L)	Chân PA11 (RX) và PA12 (TX) nối qua transceiver.
Tốc độ	500 kbps (demo này)	Prescaler=4, BS1=11, BS2=6 trên APB1 36MHz.
Khung dữ liệu	SOF + Arbitration + Control + Data + CRC + ACK + EOF	bxCAN hardware tự xử lý; phần mềm chỉ đọc payload từ FIFO.

2.2 Cấu trúc CAN Data Frame

Trường	Kích thước	Ai xử lý	Ý nghĩa
Arbitration Field	11 hoặc 29 bit	bxCAN Hardware	Chứa CAN ID, quyết định quyền ưu tiên trên bus.
Control Field	6 bit	bxCAN Hardware	Chứa DLC (Data Length Code), cho biết số byte data.
Data Field	0..8 byte	Software (COM)	Payload thực tế chứa các Signal đã được pack.
CRC + ACK	16+2 bit	bxCAN Hardware	Bảo vệ dữ liệu và xác nhận nhận thành công.

> **Điểm khác biệt lớn nhất so với LIN:** CAN dùng cơ chế *Push/Event-Driven* — Node TX gửi frame khi muốn, không cần lịch trình. Ngược lại, LIN dùng *Pull/Time-Triggered* — chỉ phát khi Master scheduler cho phép.

2.3 Phần cứng bxCAN trong STM32F103

Thành phần	Mô tả	Vai trò trong luồng RX
Filter Banks (14 bộ)	Bộ lọc phần cứng, chỉ cho frame phù hợp vào FIFO	Demo cấu hình Mask=0 → chấp nhận **tất cả** CAN ID
RX FIFO0 / FIFO1	Hàng đợi chứa frame đã lọc	ISR và Polling đọc frame từ FIFO0
TX Mailbox (3 slot)	Bộ đệm gửi	Không dùng trong luồng RX
NVIC IRQ	Interrupt cho FIFO0 message pending	`USB_LP_CAN1_RX0_IRQHandler`

---

3. Kiến trúc AUTOSAR COM Stack và vị trí luồng RX

3.1 Bốn module trên luồng RX

Trên luồng nhận (RX), dữ liệu đi **từ dưới lên trên**. Mỗi tầng có một vai trò rõ ràng:

Tầng	Module	File	Nhiệm vụ chính trong luồng RX
MCAL	Can Driver	`autosar/can/Can.c`	Đọc CAN frame từ FIFO phần cứng bxCAN, gọi `CanIf_RxIndication()`
ECU Abstraction	CanIf	`autosar/canif/CanIf.c`	Dịch CAN ID → PDU ID, gọi `PduR_CanIfRxIndication()`
Service	PduR	`autosar/pdur/PduR.c`	Tra bảng route RX, chuyển PDU đến COM qua `Com_RxIndication()`
Service	COM	`autosar/com/Com.c`	Lưu payload vào shadow buffer; khi App đọc → unpack byte thành Signal

3.2 Hai khái niệm cốt lõi: Signal vs PDU

Khái niệm	Mô tả	Ví dụ cụ thể trong demo
Signal	Giá trị logic đơn lẻ mà ứng dụng quan tâm	`Engine_RPM = 1050` (16-bit), `Engine_Temp = 70` (8-bit)
I-PDU	Interaction Layer PDU — gói byte chứa nhiều Signal	`EngineStatus` = 3 byte = [RPM_Hi, RPM_Lo, Temp]
L-PDU	Lower Layer PDU — tương đương CAN frame trên bus	CAN ID 0x181, DLC=3, Data= [0x04,0x1A,0x46]

3.3 Tại sao cần 4 tầng?

Module	Câu hỏi	Trả lời
Can Driver	Tại sao không để App đọc FIFO trực tiếp?	Vì MCAL trừu tượng hóa phần cứng. Đổi MCU chỉ cần đổi driver, code trên không thay đổi.
CanIf	Tại sao cần CanIf khi đã có Can Driver?	Vì CAN chỉ biết CAN ID (0x181), nhưng tầng trên dùng PDU ID (5). CanIf dịch giữa 2 thế giới.
PduR	Tại sao không từ CanIf gọi thẳng COM?	Vì PDU có thể đi đến COM (data thường) hoặc DCM (chẩn đoán). PduR là "bưu điện" phân phối.
COM	Tại sao cần COM khi PduR đã có data?	Vì PduR chỉ biết PDU (mảng byte). COM biết Signal — nơi unpack byte thành giá trị có ý nghĩa.

> **Điểm rất dễ nhầm:** Luồng RX chia làm **2 giai đoạn riêng biệt không đồng bộ**:
> **Giai đoạn 1 (Nhận & Lưu)**: ISR hoặc Polling → chuỗi callback → copy vào shadow buffer. Xảy ra bất cứ lúc nào có frame.
> **Giai đoạn 2 (Đọc Signal)**: App chủ động gọi `Com_ReceiveSignal()` khi cần. Xảy ra trong main loop.

---

4. Mapping trực tiếp vào code và cấu hình của repo

4.1 Cấu hình COM — Signal Layout trong I-PDU EngineStatus

I-PDU `EngineStatus` (PduId=5) chứa 2 Signal, được layout như sau:

```
Byte: [ 0 ] [ 1 ] [ 2 ]
├──────────────┤├─────┤
Signal: │ Engine_RPM ||Temp │
│ (16-bit BE) ||(8b) │
└──────────────┘└─────┘
```

Signal	COM I-PDU	Signal ID	Byte Index	Bit Length	Direction
`ComSig_Engine_RPM`	`ComConf_ComIPdu_EngineStatus = 5`	17	0	16 bit	RX
`ComSig_Engine_Temp`	`ComConf_ComIPdu_EngineStatus = 5`	18	2	8 bit	RX

Cấu hình trong `config/Com_Cfg.c` dùng `#ifdef NODE_TX` / `NODE_RX` để chuyển direction.

4.2 Cấu hình CanIf — Ánh xạ CAN ID ↔ PDU ID

CanIf PDU ID (symbolic)	Giá trị số	CAN ID	DLC Max
`CanIfConf_Pdu_EngineCmd`	0	0x180	5
`CanIfConf_Pdu_BrakeCmd`	1	0x280	3
`CanIfConf_Pdu_BodyCmd`	2	0x380	4
`CanIfConf_Pdu_DiagTx`	3	0x7DF	8
`CanIfConf_Pdu_DiagRx_FC`	4	0x7E8	8
`CanIfConf_Pdu_EngineStatus`	**5**	**0x181**	**3**

4.3 Cấu hình PduR — Routing Table RX

Bảng route	SrcPduId (từ CanIf)	DstPduId (đến COM)
`PduR_CanIfRxRoutes[0]`	`CanIfConf_Pdu_EngineStatus = 5`	`ComConf_ComIPdu_EngineStatus = 5`

4.4 Tổng hợp ánh xạ xuyên suốt

```
CAN ID (HW) PDU ID (CanIf) PDU ID (COM) Signal ID
0x181 → CanIfConf_Pdu_ → ComConf_ComIPdu_ → ComSig_Engine_RPM (17)
EngineStatus (5) EngineStatus (5) → ComSig_Engine_Temp (18)
```

---

5. Luồng thực thi đầy đủ từ CAN Bus đến Application

5.1 Giai đoạn khởi tạo

```
void BSW_Init_RX(void) {
    Can_Init();                // 1. Cấu hình bxCAN: GPIO, filter, bit t
    CanIf_Init();              // 2. Đánh dấu CanIf sẵn sàng
    PduR_Init(&PduR_ConfigPB); // 3. Load bảng route, bật routing
    Com_Init();                // 4. Xóa toàn bộ shadow buffer về 0
}
```

> **Thứ tự khởi tạo rất quan trọng:** Phải init từ **dưới lên trên** (Can → CanIf → PduR → COM), vì tầng trên phụ thuộc vào tầng dưới. Nếu đảo ngược, callback chain sẽ gọi vào module chưa sẵn sàng.

5.2 Tầng 1 — Can Driver: Đọc frame từ phần cứng

CAN Driver hỗ trợ **2 cơ chế nhận song song**:

Cơ chế	Hàm	Khi nào chạy	Ưu điểm
Interrupt (ISR)	`USB_LP_CAN1_RX0_IRQHandler()`	Tức thì khi frame vào FIFO0	Độ trễ thấp nhất
Polling	`Can_MainFunction_Read()`	Định kỳ từ main loop	Backup, đảm bảo không bỏ sót

```
// Can.c — ISR nhận CAN frame (dòng 404–417)
void USB_LP_CAN1_RX0_IRQHandler(void)
{
    CanRxMsg rxMsg;        // Cấu trúc SPL
    PduInfoType pduInfo;   // Cấu trúc AUTOSAR

    while (CAN_MessagePending(CAN_PERIPH, CAN_FIFO0) > 0) {
        CAN_Receive(CAN_PERIPH, CAN_FIFO0, &rxMsg);  // Đọc từ FIFO HW

        pduInfo.SduDataPtr = rxMsg.Data;    // Con trỏ payload
        pduInfo.SduLength  = rxMsg.DLC;     // Số byte

        CanIf_RxIndication(rxMsg.StdId, &pduInfo);  // ← Gọi lên CanIf
    }
}
```

5.3 Tầng 2 — CanIf: Dịch CAN ID → PDU ID

```
// CanIf.c — Routing theo CAN ID (dòng 162–178)
void CanIf_RxIndication(Can_IdType canId, const PduInfoType* PduInfoPtr)
{
    if (PduInfoPtr == NULL) return;

    if (canId == 0x7DF || canId == 0x7E8) {
        CanTp_RxIndication(0, PduInfoPtr);           // Diagnostic → Car
    }
    else if (canId == 0x181) {
        PduR_CanIfRxIndication(                       // EngineStatus →
            CanIfConf_Pdu_EngineStatus, PduInfoPtr);  //   PDU ID = 5
    }
}
```

Điểm mấu chốt của CanIf: Phần cứng CAN chỉ biết CAN ID (`0x181`). Tầng trên (PduR, COM) chỉ biết PDU ID (`5`). CanIf là nơi **duy nhất** thực hiện việc dịch giữa 2 thế giới ID này.

5.4 Tầng 3 — PduR: Tra bảng route, chuyển đến COM

```
// PduR.c — RX callback (dòng 182–194)
void PduR_CanIfRxIndication(PduIdType CanIfRxPduId, const PduInfoType* P
{
    if (s_State == PDUR_UNINIT || !s_RoutingEnabled) return;
    if (s_Cfg->CanIfRxRoutingTable == NULL) return;

    // Tra bảng: SrcPduId=5 → DstPduId=5
    int32_t idx = prv_find_callback_route(
        s_Cfg->CanIfRxRoutingTable,
        PDUR_NUM_CANIF_RX_ROUTES,   // = 1
        CanIfRxPduId);              // = 5

    if (idx >= 0) {
        PduIdType comPduId = s_Cfg->CanIfRxRoutingTable[idx].DstPduId;
        Com_RxIndication(comPduId, PduInfoPtr);   // ← Gọi lên COM
    }
}
```

5.5 Tầng 4 — COM: Lưu vào shadow buffer

```
// Com.c — RX Indication (dòng 302–318)
void Com_RxIndication(PduIdType ComRxPduId, const PduInfoType* PduInfoPt
{
    PduLengthType len = 0u;
    Com_PduDirection_e dir = COM_PDU_DIR_TX;
    uint8* buf = prv_get_ipdu_buf(ComRxPduId, &len, &dir);

    if ((buf != NULL) && (dir == COM_PDU_DIR_RX)) {
        PduLengthType copyLen = (PduInfoPtr->SduLength < len)
                                ? PduInfoPtr->SduLength : len;
        memcpy(buf, PduInfoPtr->SduDataPtr, copyLen);  // ← Copy vào buf
    }
}
```

```
Trước Com_RxIndication(): Com_IpduBuf[5] = [0x00, 0x00, 0x00]
Sau Com_RxIndication(): Com_IpduBuf[5] = [0x04, 0x1A, 0x46]
├──────────┤ ├─┤
RPM=0x041A Temp=0x46
RPM=1050 Temp=70
```

5.6 Application: Đọc Signal từ buffer

```
// Com.c — ReceiveSignal (dòng 254–296)
uint8 Com_ReceiveSignal(Com_SignalIdType SignalId, void* SignalDataPtr)
{
    const Com_SignalCfgType* cfg = &Com_SignalCfg[SignalId];
    uint8* ipdu = prv_get_ipdu_buf(cfg->PduId, &ipduLen, &dir);

    // Kiểm tra hướng phải là RX
    if (dir != COM_PDU_DIR_RX) return COM_SERVICE_NOT_AVAILABLE;

    switch (cfg->bitLength) {
        case 16u:
            *(uint16*)SignalDataPtr = get_u16(ipdu, cfg->byteIndex);  //
            break;
        case 8u:
            *(uint8*)SignalDataPtr = get_u8(ipdu, cfg->byteIndex);
            break;
        // ...
    }
    return E_OK;
}
```

Hàm unpack 16-bit Big Endian:

```
static inline uint16 get_u16(const uint8* src, uint16 byteIndex) {
    return (uint16)(((uint16)src[byteIndex] << 8) | (uint16)src[byt
}
// Ví dụ: get_u16([0x04, 0x1A, 0x46], 0) = (0x04 << 8) | 0x1A = 0x0
```

---

6. Phân tích ở mức byte/bit: Pack và Unpack Signal

6.1 Phía TX: Com_SendSignal pack Signal vào buffer

Node TX gọi `Com_SendSignal(ComSig_Engine_RPM, &rpm)` với `rpm = 1050`:

```
put_u16(buf, 0, 1050)
→ buf[0] = (uint8)(1050 >> 8) = (uint8)(0x041A >> 8) = 0x04
→ buf[1] = (uint8)(1050 & 0xFF) = (uint8)(0x041A & FF) = 0x1A
```

Node TX gọi `Com_SendSignal(ComSig_Engine_Temp, &temp)` với `temp = 70`:

```
put_u8(buf, 2, 70)
→ buf[2] = 0x46
```

Kết quả buffer I-PDU trước khi gửi: `[0x04, 0x1A, 0x46]`

6.2 Phía RX: Com_ReceiveSignal unpack buffer ra Signal

Signal	Hàm unpack	Tính toán	Kết quả
`Engine_RPM`	`get_u16(buf, 0)`	`(0x04 << 8) | 0x1A = 0x041A`	**1050**
`Engine_Temp`	`get_u8(buf, 2)`	`0x46`	**70**

---

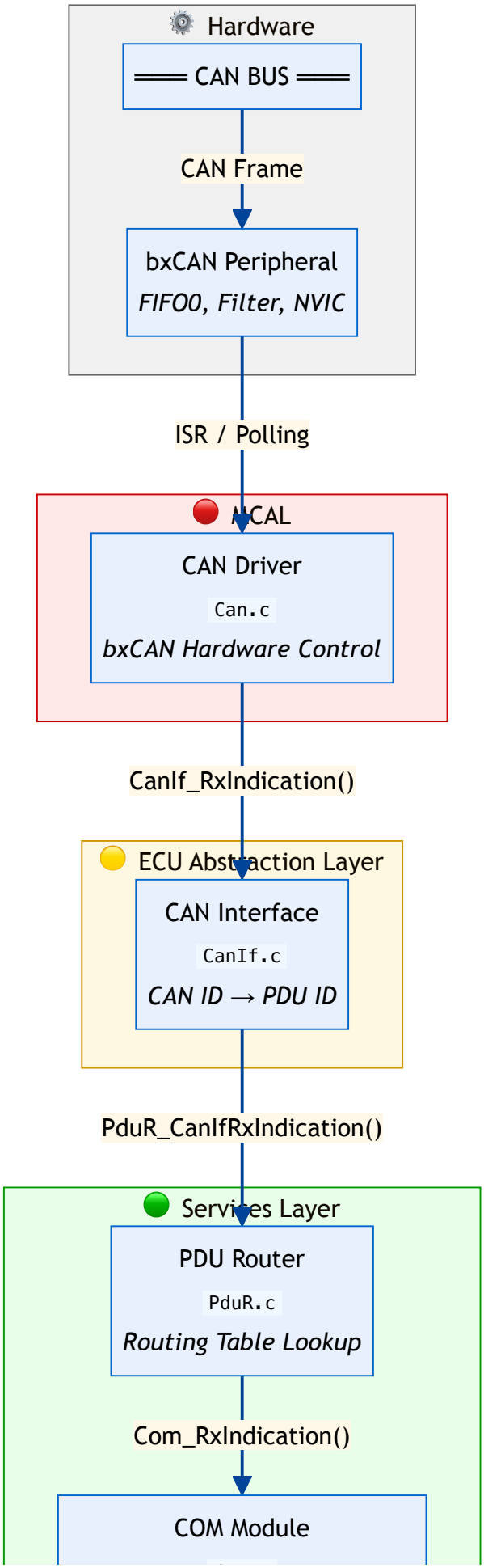
7. Timeline thực tế — Theo dõi 1 CAN frame xuyên suốt

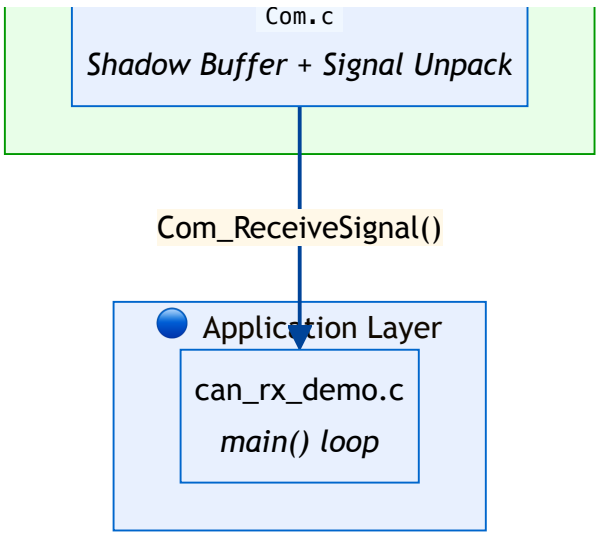
Giả sử Node TX gửi `RPM=1050, Temp=70`. Bảng sau mô tả sự kiện xảy ra tại mỗi tầng:

Bước	Vị trí	Sự kiện	Dữ liệu
1	TX Node – COM	`Com_SendSignal()` pack RPM, Temp vào buffer	buf = [0x04, 0x1A, 0x46]
2	TX Node – PduR	`Com_TriggerIPDUSend(5)` → `PduR_ComTransmit()`	PduInfo: ptr=buf, len=3
3	TX Node – CanIf	`CanIf_Transmit(5)` tra config: ID=0x181, HTH=5	Can_PduType: id=0x181, DLC=3
4	TX Node – Can	`Can_Write(5, &pdu)` → `CAN_Transmit()`	CAN frame nạp vào mailbox
5	**CAN BUS**	Frame truyền qua Hub ảo (Renode)	ID=0x181, DLC=3, [04,1A,46]
6	RX Node – Can	ISR: `CAN_Receive(FIFO0)`	rxMsg.StdId=0x181, Data= [04,1A,46]
7	RX Node – CanIf	`CanIf_RxIndication(0x181, &pduInfo)`	Dịch 0x181 → PDU ID 5
8	RX Node – PduR	`PduR_CanIfRxIndication(5, &pduInfo)`	Route tìm thấy: 5 → 5
9	RX Node – COM	`Com_RxIndication(5, &pduInfo)`	`memcpy(shadow_buf, [04,1A,46], 3)`
10	RX Node – App	`Com_ReceiveSignal(17, &rpm)`	rpm = get_u16(buf,0) = **1050**
11	RX Node – App	`Com_ReceiveSignal(18, &temp)`	temp = get_u8(buf,2) = **70**

8. Sơ đồ UML chi tiết

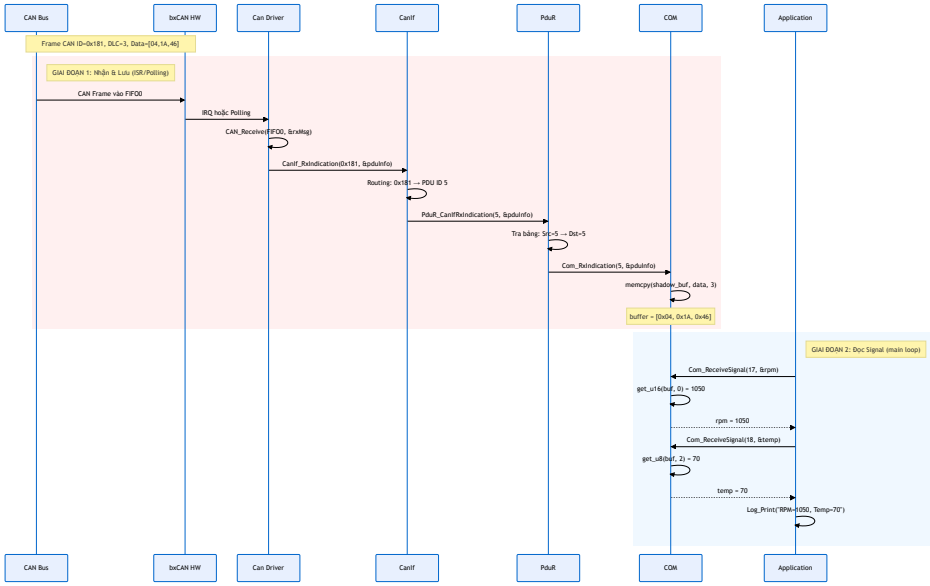
Hình 1. Component Diagram — Kiến trúc lớp AUTOSAR CAN RX


Hardware
CAN BUS
CAN Frame
bxCAN Peripheral
FIFO0, Filter, NVIC
ISR / Polling
MCAL
CAN Driver
Can.c
bxCAN Hardware Control
CanIf_RxIndication()
ECU Abstraction Layer
CAN Interface
CanIf.c
CAN ID → PDU ID
PduR_CanIfRxIndication()
Services Layer
PDU Router
PduR.c
Routing Table Lookup
Com_RxIndication()
COM Module



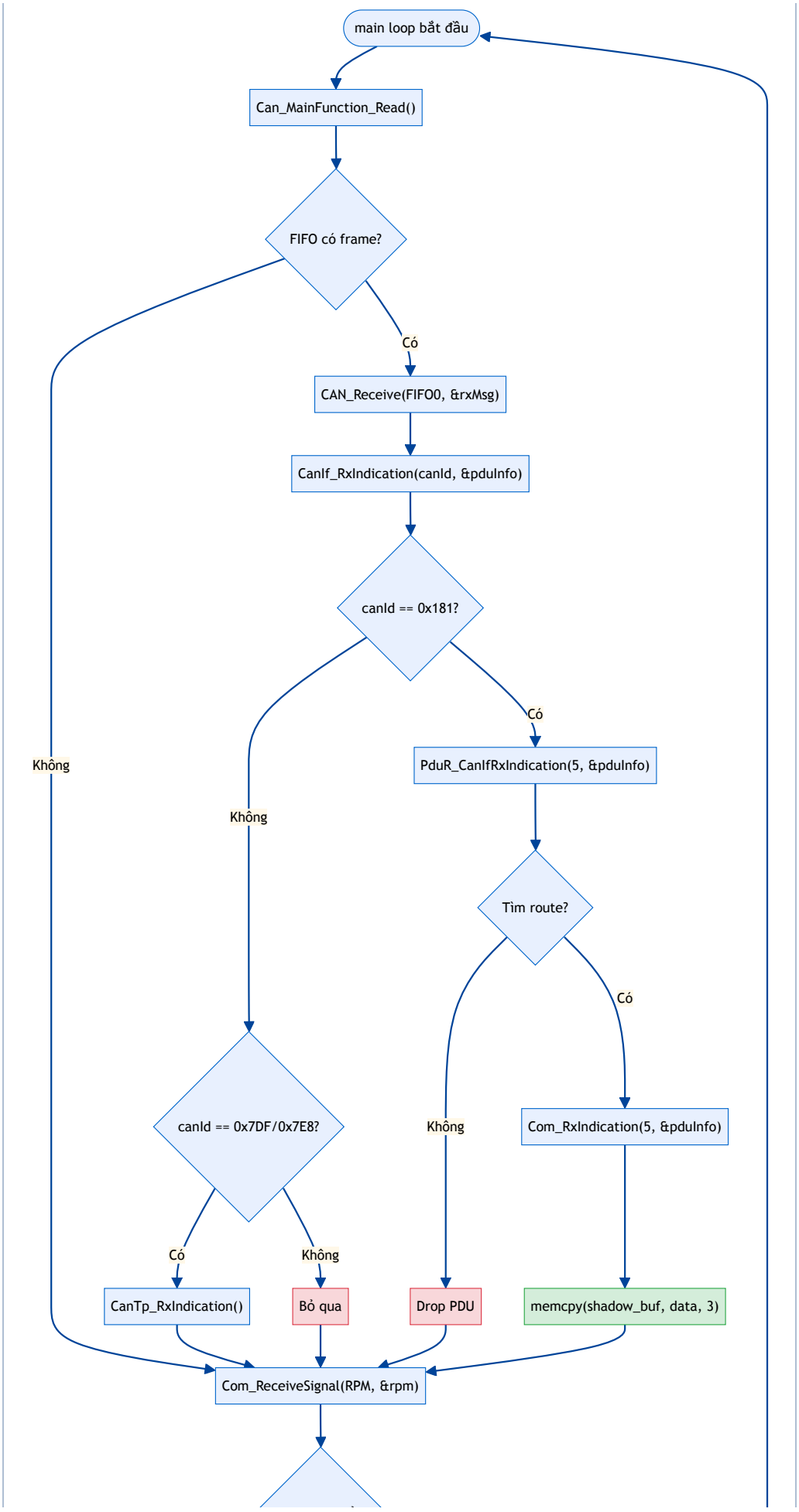
Luồng RX: CAN Bus → bxCAN → Can Driver → CanIf → PduR → COM → Application

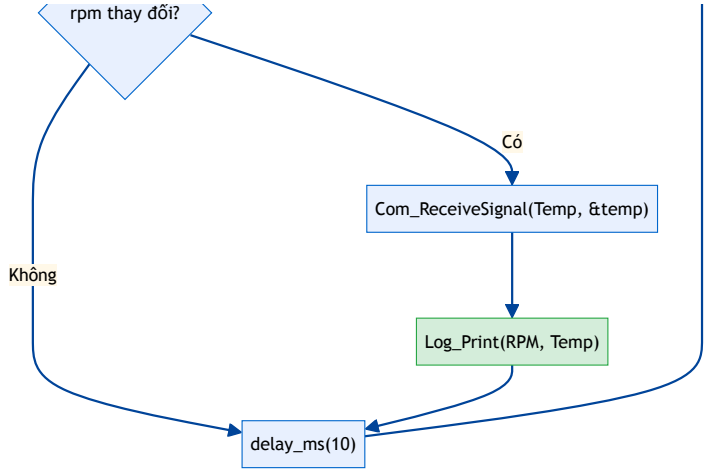
Hình 2. Sequence Diagram — Luồng nhận CAN frame hoàn chỉnh



Hai giai đoạn không đồng bộ: ISR/Polling trigger chuỗi callback (hồng), App đọc Signal khi muốn (xanh).

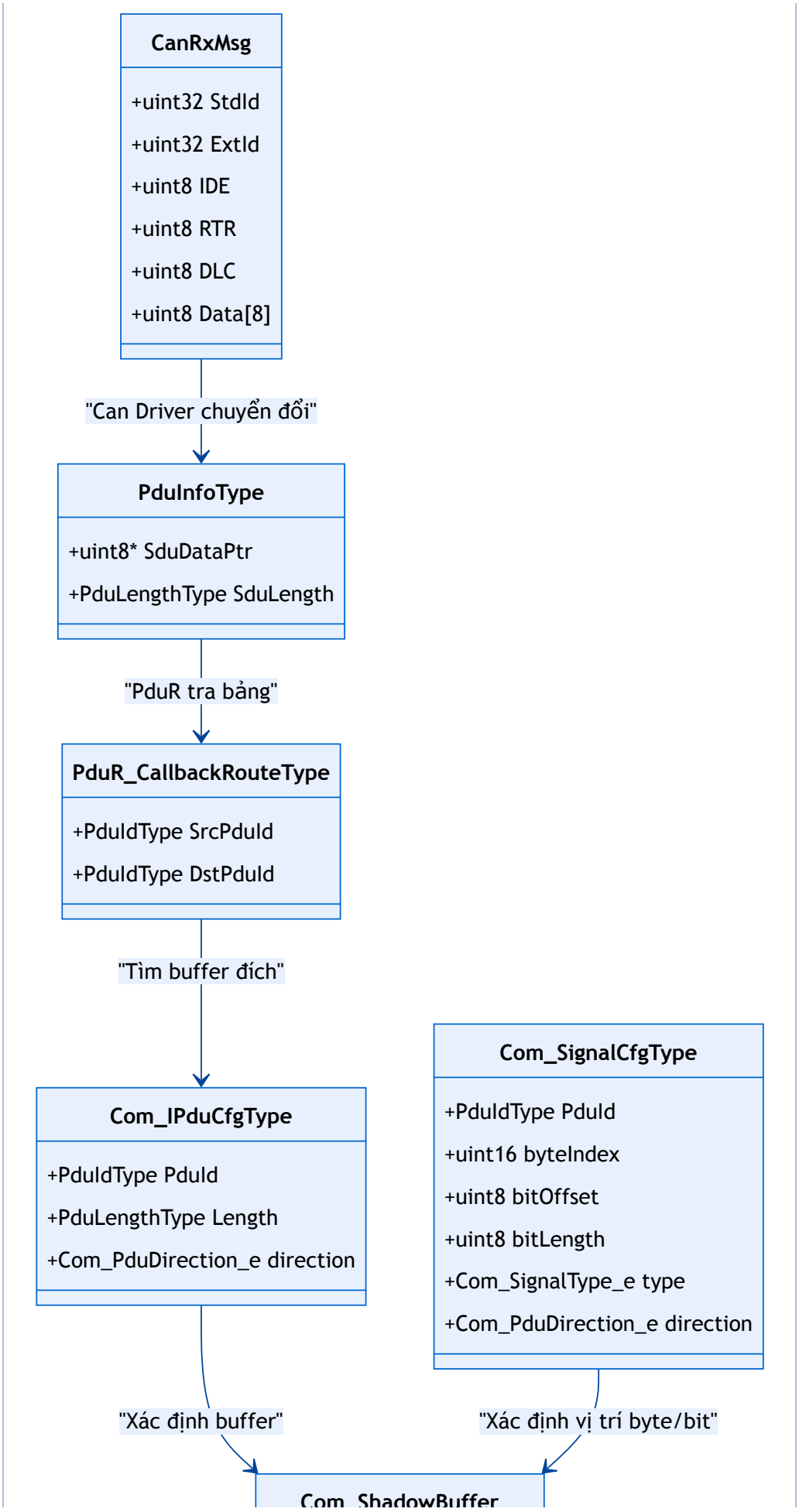
Hình 3. Activity Diagram — Luồng xử lý trong main() Node RX

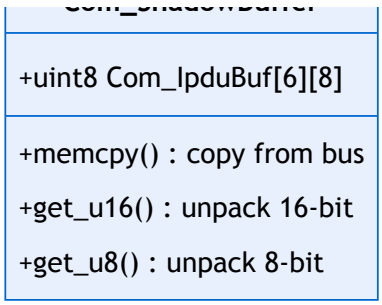

main loop bắt đầu
Can_MainFunction_Read()
FIFO có frame?
Có
Không
CAN_Receive(FIFO0, &rxMsg)
CanIf_RxIndication(canId, &pduInfo)
canId == 0x181?
Có
Không
PduR_CanIfRxIndication(5, &pduInfo)
Tìm route?
Có
Không
Com_RxIndication(5, &pduInfo)
memcpy(shadow_buf, data, 3)
canId == 0x7DF/0x7E8?
Có
Không
CanTp_RxIndication()
Bỏ qua
Drop PDU
Com_ReceiveSignal(RPM, &rpm)



Logic hoàn chỉnh của vòng lặp main() trên Node RX, bao gồm cả nhánh polling, routing và đọc Signal.

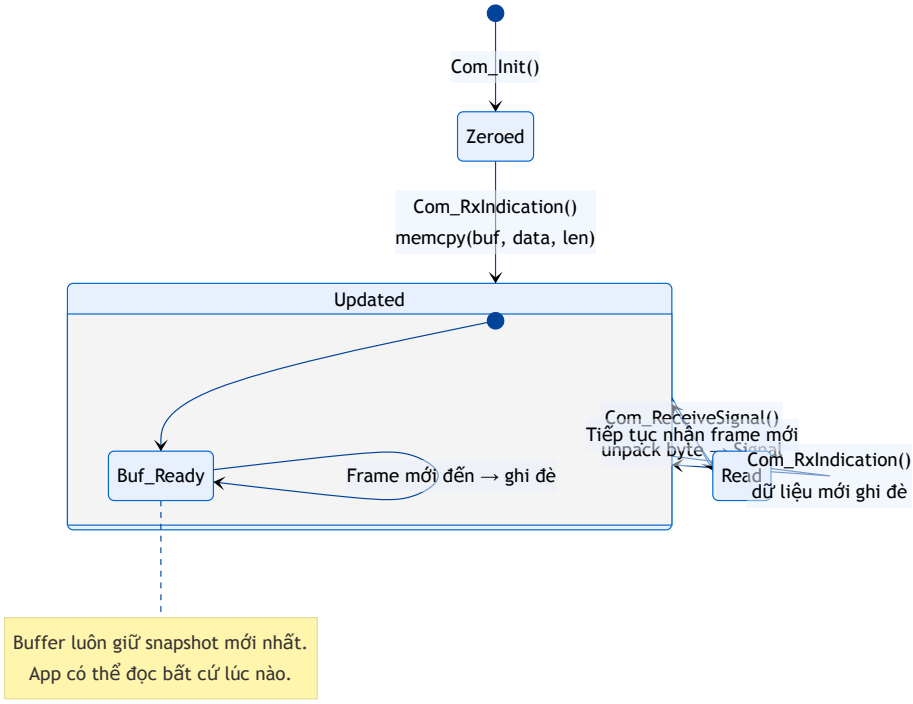
Hình 4. Class Diagram — Cấu trúc dữ liệu chính trên luồng RX


CanRxMsg
+uint32 StdId
+uint32 ExtId
+uint8 IDE
+uint8 RTR
+uint8 DLC
+uint8 Data[8]
"Can Driver chuyển đổi"
PduInfoType
+uint8* SduDataPtr
+PduLengthType SduLength
"PduR tra bảng"
PduR_CallbackRouteType
+PduIdType SrcPduId
+PduIdType DstPduId
"Tìm buffer đích"
Com_IPduCfgType
+PduIdType PduId
+PduLengthType Length
+Com_PduDirection_e direction
Com_SignalCfgType
+PduIdType PduId
+uint16 byteIndex
+uint8 bitOffset
+uint8 bitLength
+Com_SignalType_e type
+Com_PduDirection_e direction
"Xác định buffer"
"Xác định vị trí byte/bit"
Com_ShadowBuffer



Quan hệ giữa các cấu trúc dữ liệu trên luồng RX: từ CanRxMsg (SPL) → PduInfoType (AUTOSAR) → Shadow Buffer → Signal.

Hình 5. State Diagram — Trạng thái Shadow Buffer trong COM

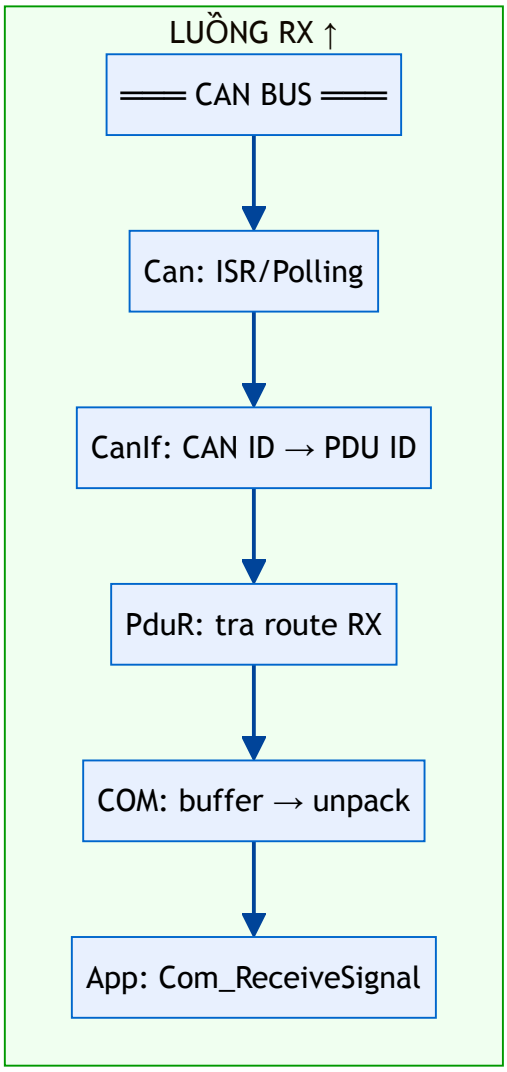


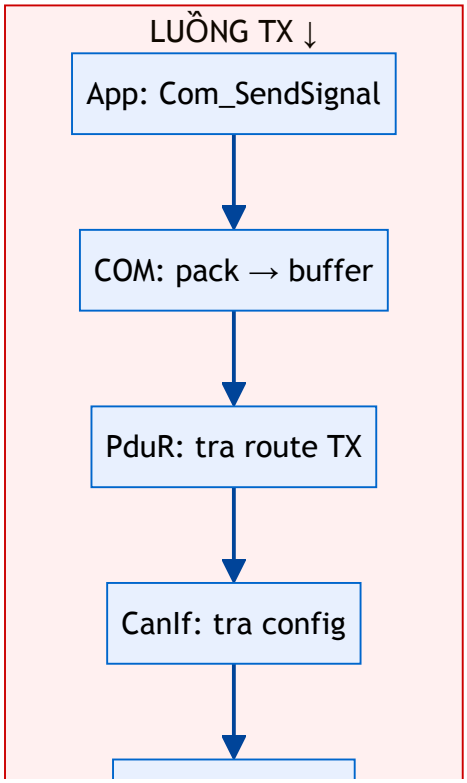
Shadow buffer bắt đầu từ trạng thái Zeroed (sau Com_Init), chuyển sang Updated mỗi khi có frame mới, và App đọc bất kỳ lúc nào.

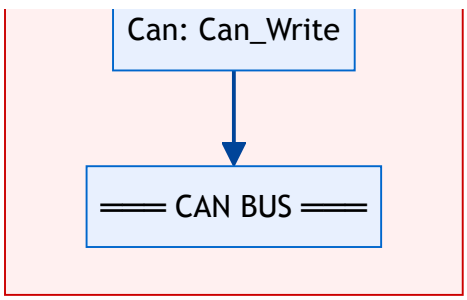
9. So sánh luồng TX và RX

Đặc điểm	Luồng TX (Gửi)	Luồng RX (Nhận)
Hướng dữ liệu	Trên → Dưới (App → Bus)	Dưới → Trên (Bus → App)
Trigger	App gọi `Com_SendSignal()`	ISR hoặc `Can_MainFunction_Read()`
COM làm gì	Pack Signal → I-PDU buffer	Copy I-PDU vào buffer + Unpack khi đọc
PduR làm gì	Tra bảng TX route → gọi `CanIf_Transmit()`	Tra bảng RX route → gọi `Com_RxIndication()`
CanIf làm gì	Tra bảng config → `Can_Write()`	CAN ID → PDU ID → `PduR_CanIfRxIndication()`
Can Driver	Nạp frame vào mailbox TX	Đọc frame từ FIFO RX
Cơ chế gọi	API call (đồng bộ, push)	Callback chain (bất đồng bộ, từ ISR)

Hình 6. So sánh trực quan TX vs RX


LUỒNG RX ↑
═══ CAN BUS ═══
Can: ISR/Polling
CanIf: CAN ID → PDU ID
PduR: tra route RX
COM: buffer → unpack
App: Com_ReceiveSignal


LUỒNG TX ↓
App: Com_SendSignal
COM: pack → buffer
PduR: tra route TX
CanIf: tra config



TX đi từ trên xuống (Push), RX đi từ dưới lên (Callback chain).

---

10. Kiểm chứng bằng log và lab mô phỏng

10.1 Chạy mô phỏng trên Renode

```
# Biên dịch
make clean && make build/example/can_tx_demo.elf build/example/can_rx_de

# Chạy Renode với 2 Node kết nối qua CAN Hub
/Applications/Renode.app/Contents/MacOS/Renode --disable-gui \
    -e "s @scripts/stm32_com_verify.resc" -e "start"
```

10.2 Kết quả log thực tế

```
=== TX LOG (Engine ECU) ===
--- Engine ECU (Node TX) Booted ---
TX: Sending EngineStatus -> RPM=1050, Temp=71
TX: Sending EngineStatus -> RPM=1100, Temp=72
TX: Sending EngineStatus -> RPM=1150, Temp=73
TX: Sending EngineStatus -> RPM=1200, Temp=74
...

=== RX LOG (VCU ECU) ===
--- VCU ECU (Node RX) Booted ---
RX: Received EngineStatus -> RPM=0, Temp=0
RX: Received EngineStatus -> RPM=1050, Temp=71
RX: Received EngineStatus -> RPM=1100, Temp=72
RX: Received EngineStatus -> RPM=1150, Temp=73
RX: Received EngineStatus -> RPM=1200, Temp=74
...
```

Xác nhận thành công: Giá trị RPM và Temp trên Node RX khớp chính xác với Node TX, với độ trễ 1 cycle (do polling). Giá trị đầu tiên `RPM=0, Temp=0` là giá trị khởi tạo trước khi frame đầu tiên đến.

10.3 Điểm quan sát khi debug

Vị trí đặt breakpoint	Biến cần watch	Mục đích
`CanIf_RxIndication()`	`canId`, `PduInfoPtr->SduLength`	Xác nhận frame đúng CAN ID và DLC
`PduR_CanIfRxIndication()`	`CanIfRxPduId`, `idx`	Xác nhận route được tìm thấy
`Com_RxIndication()`	`buf`, `copyLen`	Xác nhận data copy đúng vào buffer
`Com_ReceiveSignal()`	`SignalId`, `*SignalDataPtr`	Xác nhận unpack đúng giá trị

---

11. Các điểm dễ nhầm và kết luận kỹ thuật

11.1 Năm điểm dễ nhầm nhất

- `Com_ReceiveSignal()` **không kích hoạt việc nhận.** Nó chỉ đọc từ buffer đã có sẵn. Việc nhận xảy ra ở chuỗi callback ISR/Polling.
- **CAN ID ≠ PDU ID ≠ Signal ID.** Trong demo trùng nhau (đều = 5) là do thiết kế đơn giản, nhưng trong hệ thống thật chúng khác nhau.
- **Shadow buffer ghi đè không có lock.** Demo không dùng mutex — nếu ISR đang copy mà App đang đọc, có thể đọc dữ liệu "nửa cũ nửa mới". Production code cần SchM critical section.
- `Com_SendSignal()` **chỉ dành cho TX,** `Com_ReceiveSignal()` **chỉ cho RX.** Nếu gọi sai hướng, hàm trả về `COM_SERVICE_NOT_AVAILABLE`.
- **ISR và Polling cùng gọi** `CanIf_RxIndication()`**.** Chúng không xung đột vì `CAN_Receive()` dequeue frame khỏi FIFO — mỗi frame chỉ được xử lý một lần duy nhất.

11.2 Kết luận cuối cùng

Điều quan trọng nhất để hiểu CAN RX trong AUTOSAR Classic là: **Application không bao giờ tiếp xúc trực tiếp với phần cứng CAN**. Thay vào đó:

- `Can Driver` đọc frame từ FIFO phần cứng và chuyển thành cấu trúc phần mềm.
- `CanIf` dịch CAN ID thành PDU ID — cầu nối giữa thế giới phần cứng và phần mềm.
- `PduR` định tuyến PDU đến đúng module đích (COM cho data, DCM cho diagnostic).
- `COM` lưu payload vào shadow buffer và unpack thành Signal khi App yêu cầu.

Nắm được chuỗi này thì việc đọc tiếp các chủ đề nâng cao như `COM notification callback`, `signal gateway`, `deadline monitoring` hay tích hợp với OS task sẽ dễ hơn rất nhiều.